



State of Open-Source Collaboration in the Agentic Era

Across 143 landscape projects, 100 high-activity repositories and 5,000 public Issues and pull requests, this report follows how agents are entering open-source software. The map is filling in around runtimes, tools, isolation and evidence. On GitHub, Agents seldom open the work; they appear later in review and revision, while maintainers still make most visible decisions about what merges or closes.

Research frame

143 landscape projects	100 high-activity repositories
5,000 public Issues and pull requests	8 months 1 January–31 August 2026

Contents

01	State of Open-Source Collaboration in the Agentic Era	Chapter
02	01 · Landscape and Open Infrastructure	Chapter
03	01A · The current maps	Section
04	01B · Signals in the map	Section
05	01C · Open infrastructure	Section
06	02 · Open-source Collaboration	Chapter
07	02A · Workload and repository setting	Section
08	02B · Repository access and Agent setup	Section
09	02C · Where Agents enter the public workflow	Section
10	Method and evidence boundaries	Chapter
11	References	Chapter

What this report finds

Runtime work is catching up with the applications people already use.

Applications hold 55% of Agent Infra's July OpenRank. Runtime accounts for 13 of the 23 Agent Infra selections absent from the May tracking pool, filling in around context, interoperability, tool control and execution.

Repositories prepare Agents to contribute, but Agents rarely open the work.

Ninety-two of the Top 100 publish a coding-agent file or folder. A named Agent account or App opened 87 of the 5,000 sampled threads, while Agent participation appeared in 2,158. Most of the public trace enters later, through review, discussion, triage or code revision.

Code supply expanded faster than the merge path.

Across the same 55 repositories, PR intake rose from 129,563 in 2025 to 265,447 in 2026. The share still open after 90 days rose from 5.5% to 11.3%, while the repository-median merge rate fell from 77.0% to 68.4%. The public integration circle widened from 17 to 25 PushEvent accounts per repository, but it did not grow as fast as the PR queue.

Research frame

- 227 repositories in the May 2026 tracking pool;
- 277 repositories in the current canonical project list;
- 143 projects selected for the current Agent Infra and Model Infra maps;
- 84 Agent Infra projects and 59 Model Infra projects;
- 31 current selections were not present in the May tracking pool.

These counts define the project universe used in the landscape analysis. They are sample boundaries for the findings that follow.

Landscape and Open Infrastructure

The landscape is the starting point for this study. It shows where open-source work is accumulating before we examine what agents ask of production infrastructure and how they change development collaboration. The current selection contains 143 repositories: 84 in Agent Infra and 59 in Model Infra. They are drawn from a 277-repository canonical list and compared with a 227-repository tracking pool preserved in May 2026.

The two maps describe different parts of the system. Agent Infra covers applications, development frameworks and the runtime services an agent uses while completing a task. Model Infra covers model serving, training, data and compute. A place on either map means the project passed the report's editorial and activity review for that part of the ecosystem.

01A • The current maps

Agent applications lead the activity. Runtime is where the map is filling in

The current maps contain 84 Agent Infra and 59 Model Infra projects. Applications hold 55% of Agent Infra's July OpenRank, while Runtime accounts for 13 of the 23 Agent Infra selections outside the May tracking pool. Model Infra remains an older, Python-led systems base, with Serving holding 44% of its July OpenRank. The findings below follow recent growth, project age, primary language and the runtime path from context to evidence.

Start with the two current maps

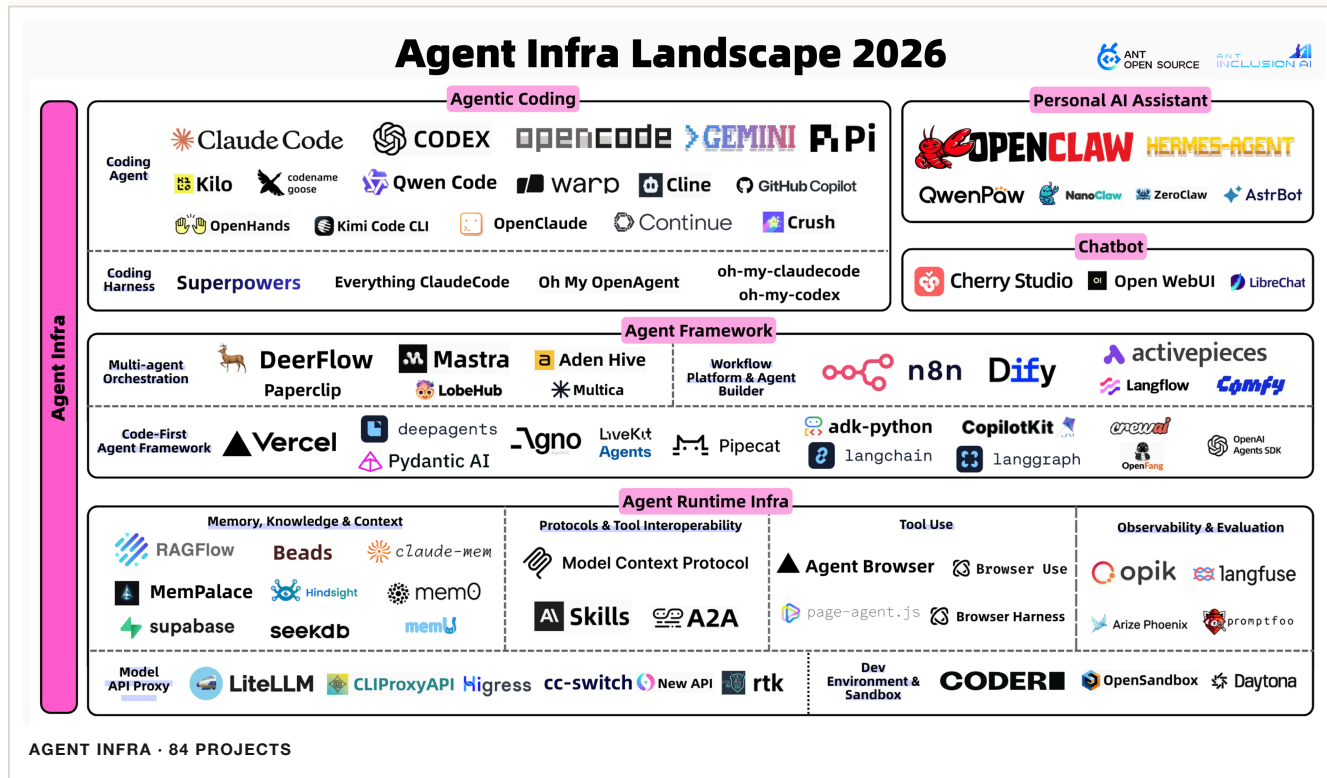
The Agent Infra map is much younger. Forty-six of its 84 projects were created in 2025 or later, and 23 were absent from the May tracking pool. Its July OpenRank leaders were OpenClaw, Hermes Agent, Deer Flow, Lark CLI and OpenViking.

Model Infra is more established. Ten of its 59 projects were created in 2025 or later, and eight were absent from the May pool. PyTorch, SGLang, vLLM, Ollama and FlashInfer led the selected projects by July OpenRank.

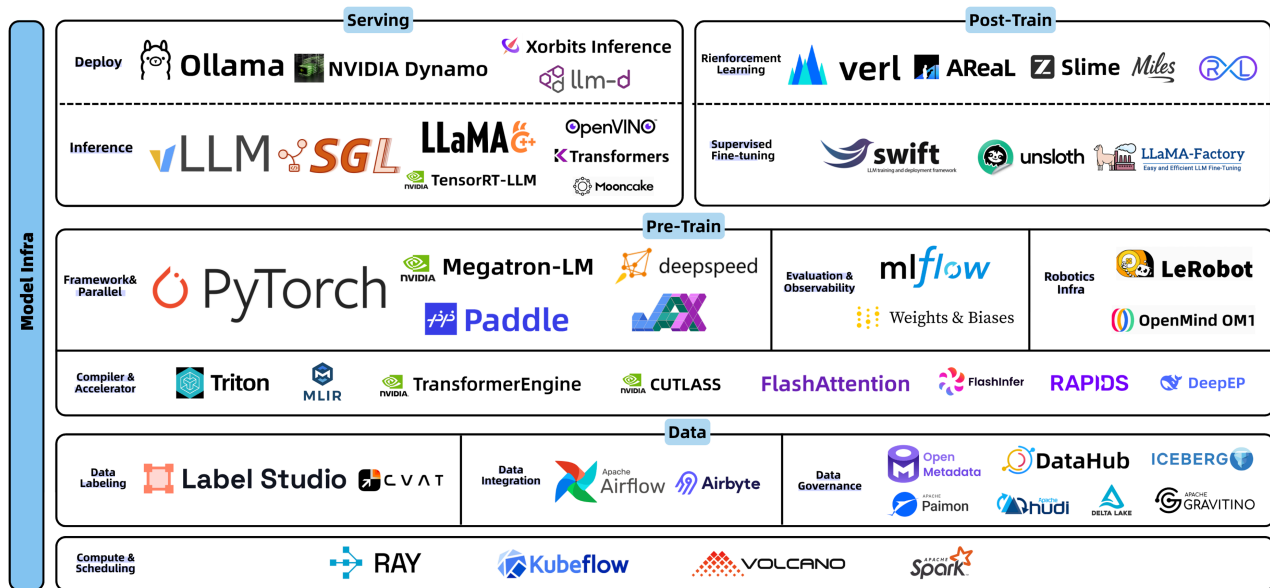
These leader lists use July 2026 [OpenRank](#), which combines repository contribution and engagement signals into a monthly project score. They show where open-source activity is concentrated inside the selected landscape.

FIGURE 01

Agent Infra and Model Infra Landscape 2026



Model Infra Landscape 2026



MODEL INFRA · 59 PROJECTS

Landscape	Selected projects	Created in 2025 or later	Five July OpenRank leaders
Agent Infra	84	46 (55%)	OpenClaw · Hermes Agent · Deer Flow · Lark CLI · OpenViking
Model Infra	59	10 (17%)	PyTorch · SGLang · vLLM · Ollama · FlashInfer

The interactive report presents the complete Agent Infra and Model Infra maps at this point, before the analytical findings. Readers can switch between them and inspect each selected repository's section, GitHub

metadata and July OpenRank. The summary below each map shows its share of projects created since 2025 and the share of July OpenRank held by its five activity leaders.

01B · Signals in the map

Applications hold the activity. Runtime holds more of the new selections

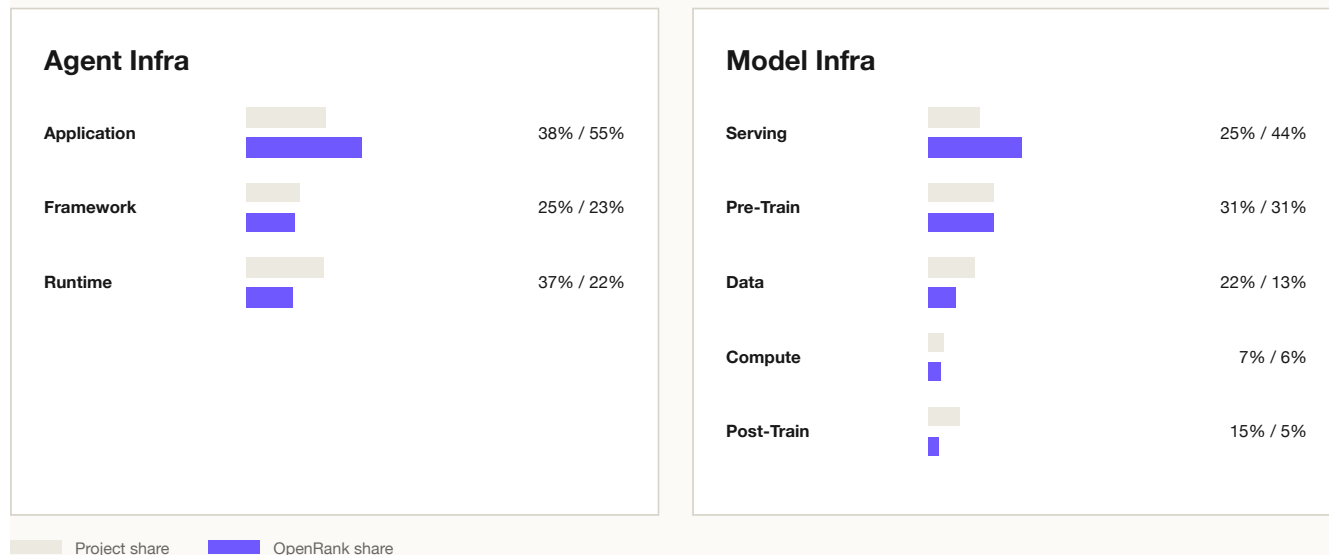
Since May, ongoing ecosystem review has expanded the tracked pool from 227 to 277 repositories. Applications still hold most of the visible Agent Infra activity: 32 projects account for 55% of the layer's combined July OpenRank. Runtime has almost the same number of selected projects, but a much smaller share of activity. It contains 31 projects and 22% of Agent Infra OpenRank.

Projects entered the tracking pool through activity-based discovery and targeted GitHub searches. A second editorial review decides which tracked projects belong on the published landscape.

The newer selections are concentrated lower in the stack. Runtime accounts for 13 of the 23 Agent Infra projects that were not in the May tracking pool. Application accounts for seven and Framework for three. The pattern suggests that ecosystem coverage is filling in around context, tool control, interoperability and execution, even while attention remains concentrated in products close to users.

FIGURE 02

Selected projects and July OpenRank by layer



Model Infra has a similar concentration of activity. Serving and Pre-Train together hold 75% of its July OpenRank. Serving alone contributes 44%, led by inference projects such as SGLang, vLLM, Ollama and FlashInfer.

The model stack may be older, but serving remains the part carrying the most visible systems work.

Coding is the first large field test for delegated software work

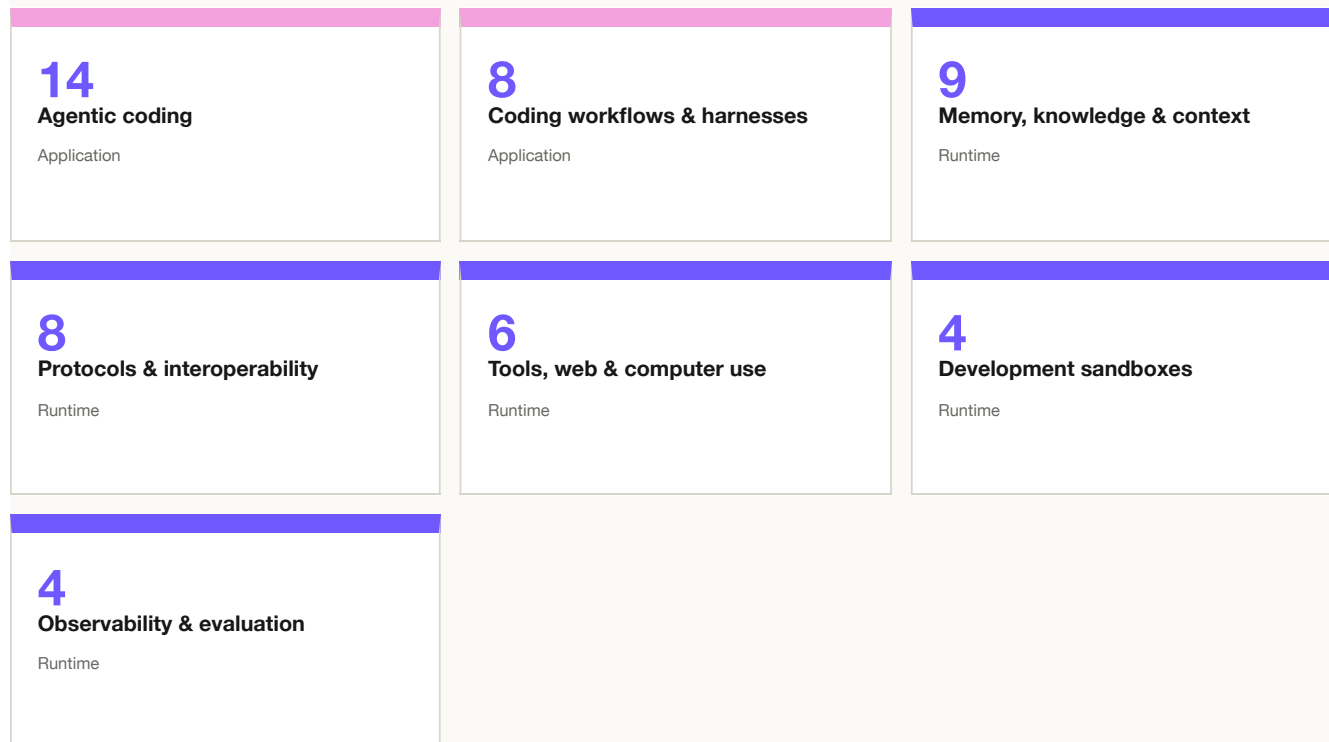
The current map contains 14 Agentic coding projects and eight coding workflows or harnesses. This is the largest application cluster. Code gives an agent a defined workspace, executable tests, a visible diff and a review path. Few other agent use cases offer such a complete feedback loop.

The rest of the map shows why coding is only the entry point. Nine selected projects manage memory, knowledge or context; eight focus on protocols and interoperability; six provide tool, web or computer use. Four projects provide development sandboxes and another four focus on observability and evaluation. Once an agent can edit code, the surrounding system has to decide what context it may use, which tools it can call, where the work can run and what evidence survives afterward.

DeepSeek Harness also exposes a governance distinction that the next chapter studies directly. Publishing source code, accepting outside changes and supporting a wider plugin ecosystem are separate choices.

FIGURE 03

Coding entry points and the adjacent Runtime stack

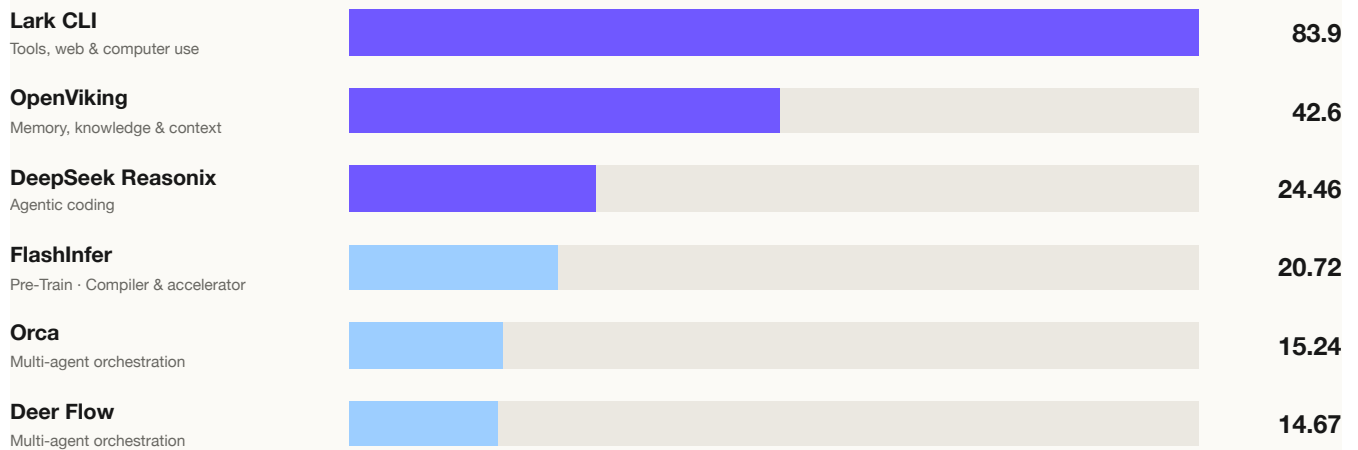


Recent activity is appearing around tools, context and inference efficiency

The strongest positive OpenRank changes between April and July did not all come from the most crowded categories. Lark CLI gained 83.90 points in Tools, web & computer use. OpenViking gained 42.60 in Memory, knowledge & context. DeepSeek Reasonix, FlashInfer, Orca and Deer Flow completed the six largest increases in the selected landscape.

FIGURE 04

Largest April-to-July OpenRank increases



These are absolute OpenRank point changes from April to July, calculated on complete monthly scores. They identify the projects whose repository activity accelerated most over the period.

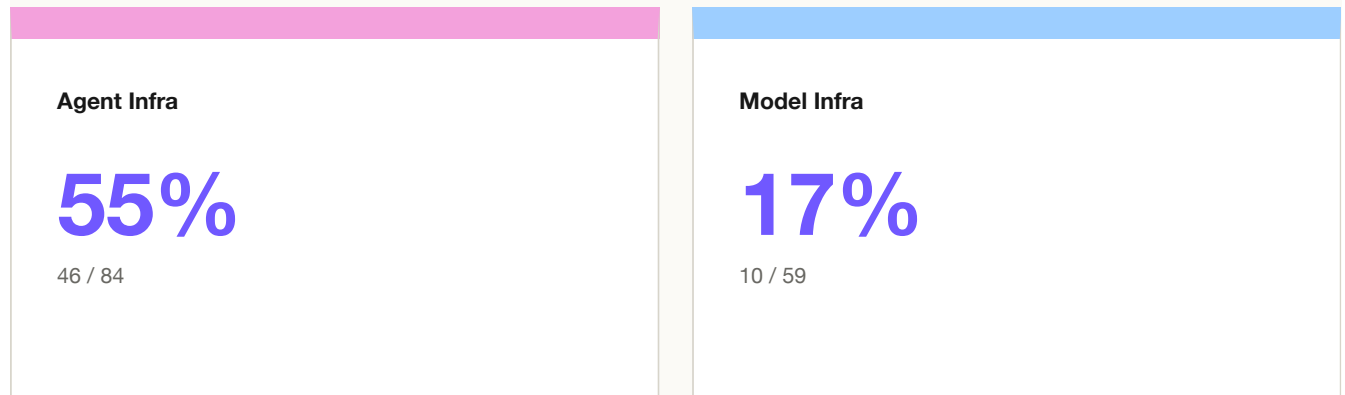
The agent layer is young. The infrastructure below it is not

Forty-six of the 84 selected Agent Infra projects were created in 2025 or later, compared with ten of 59 Model Infra projects. That is 55% of Agent Infra and 17% of Model Infra.

The age split is visible in the engineering questions each map carries. Agent interfaces and runtimes are being designed during the current wave. Model serving engines, training frameworks, schedulers and data systems bring years of existing engineering practice. Agent workloads are now asking that established base to handle short-lived code, delegated tool access and state that may outlive a process.

FIGURE 05

Age of selected projects

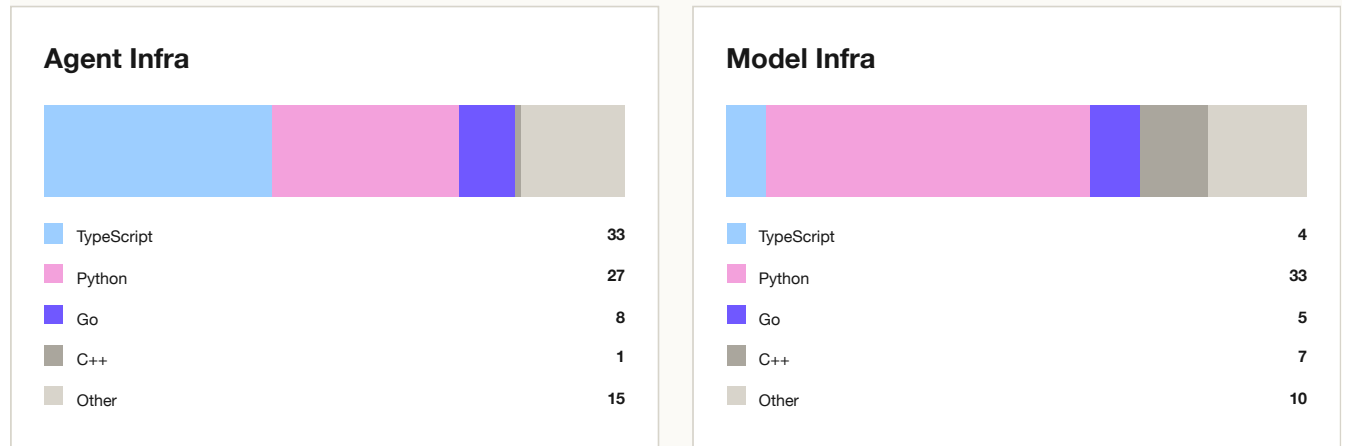


Agent products lean TypeScript. Model infrastructure still speaks Python

TypeScript is the primary language for 33 of the 84 Agent Infra repositories. Python leads 33 of the 59 Model Infra repositories. OpenClaw, Dify and Vercel AI SDK sit close to product interfaces and developer workflows; vLLM, PyTorch and SGLang remain anchored in the Python-centred model stack.

FIGURE 06

GitHub primary language of selected repositories



The field is GitHub's repository-level primary-language label. It gives each repository one comparable language category; multilingual codebases still appear under the single language GitHub marks as primary.

Runtime projects follow the path an agent takes through a task

The 31 Runtime projects can be read as an execution path. An agent retrieves context, crosses an interface, calls a tool, runs the work in an isolated environment and leaves evidence that someone can inspect later.

FIGURE 07

The Agent Runtime path



This sequence connects the selected Runtime projects to the path of an Agent task. At each step, an application concern becomes an infrastructure responsibility: context needs a lifecycle, interfaces need policy, tool calls

need scoped authority, generated code needs isolation and the resulting effect needs a durable trace.

What the map suggests next

User-facing applications still hold the largest share of Agent Infra activity, while newer selections are filling in the Runtime layer. Agent Infra is also much younger and more TypeScript-heavy than Model Infra. The next question is whether the same workloads appear in platform traffic and which established infrastructure projects are adapting to carry them.

“Outside the May pool” means a repository was absent from the preserved 227-project tracking list. It identifies an expansion in what the landscape tracks, which can include both new repositories and older projects newly brought into scope. OpenRank uses the complete July 2026 month; stars use the canonical snapshot updated on 23 August, and contributor counts were refreshed on 27 August.

01C • Open infrastructure

Agent execution is variable, stateful and capable of side effects

An agent can generate code after a task starts, fan out across model and tool calls, pause, retry and change an external system. Some runs are short. Others wait on a person or a remote service. Public token totals show that traffic exists, but they do not reveal task-level fan-out, peak concurrency or QPS. The stable infrastructure problem is broader: the task has to keep its isolation, authority, budget, state and evidence while its processes come and go.

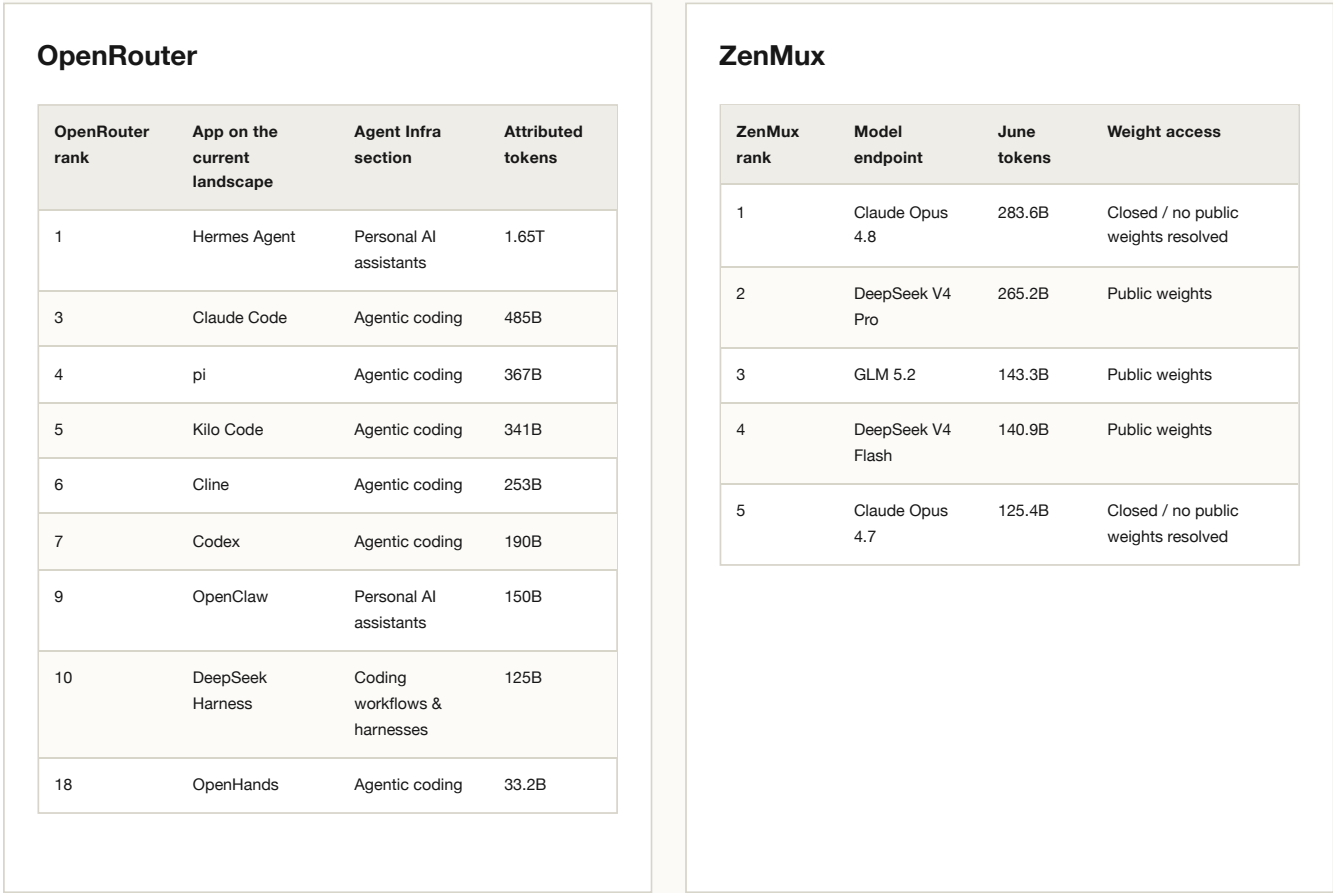
This is where the current landscape meets established open infrastructure. We check it in two places: platform traffic shows whether applications on the map are also being called, while project documentation shows which parts of the cloud-native and OpenInfra stack are being extended for Agent work.

The same workload is visible beyond repository activity

Nine of the twenty applications in OpenRouter's public, opt-in [global ranking](#) on 29 August 2026 map directly to the current Agent Infra landscape. Seven are in the Top 10. The overlap is not confined to one launch: it includes personal agents, coding tools and a coding harness.

FIGURE 08

Platform traffic outside GitHub



OpenRouter attributes traffic to public apps that opt into its ranking. Nine of its Top 20 apps on 29 August map directly to the Agent Infra landscape, including seven of the Top 10. The overlap gives the repository map an independent usage signal: several of the applications attracting open-source activity are also generating visible platform traffic.

ZenMux provides a second platform view. In its frozen export for 1–30 June 2026, three of the four most-used model endpoints linked to official public-weight repositories: DeepSeek V4 Pro, GLM 5.2 and DeepSeek V4 Flash.

Claude Opus 4.8 ranked first. OpenRouter points to demand around Agent applications; ZenMux shows open models carrying a substantial share of model traffic on a separate platform.

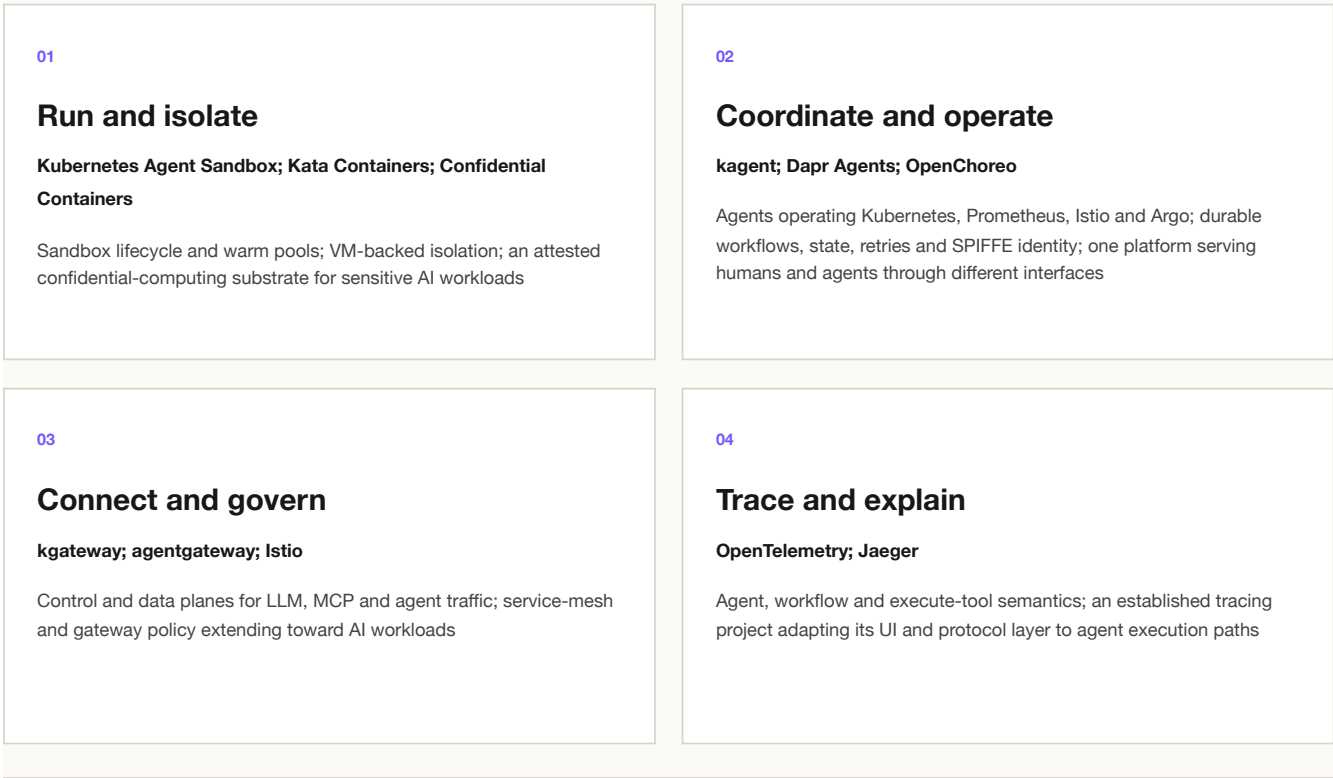
ZenMux now exposes [app](#) and [model](#) leaderboards with daily aggregation. The June values above come from the study's frozen platform export and should be refreshed before publication. OpenRouter and ZenMux token totals are never added together.

Open infrastructure is taking on the task in several places

The response is broader than sandboxing. Eleven projects provide direct or adjacent evidence across four jobs. The table distinguishes projects built explicitly for agents from established infrastructure adapting to them.

FIGURE 09

Open infrastructure projects around the task



Confidential Containers contributes an AI-relevant isolation substrate, while Istio adapts an established service mesh to AI traffic. Prometheus and Argo appear because kagent ships tools that operate them, showing how Agents

are becoming consumers of the existing stack as well as users of new Agent-specific projects.

A production agent needs a task envelope

Agent behaviour	Established open infrastructure	Work still open
Start an environment and run unknown code	Kubernetes lifecycle, Agent Sandbox and Kata isolation	strong isolation with low startup latency; safe warm-pool reset and portable profiles
Call models, APIs and MCP tools	gateways, service mesh and agentgateway	task-wide budgets, fan-out limits, backpressure and cancellation across the whole chain
Pause, resume or retry long work	Dapr workflows, messaging and state systems	checkpoints, idempotency and compensation after a tool has already produced an effect
Borrow authority for one task	SPIFFE/SPIRE workload identity and policy systems	delegation bound to user intent, tools, resources, approval state and expiry
Carry context across processes	databases, object storage and context projects	lineage, TTL, deletion, inheritance and recovery from poisoned context
Change an external system	OpenTelemetry and Jaeger trace pipelines	causal evidence from model work to tool execution and the external result
Mix inference, tools and sandbox compute	Kubernetes DRA and Kueue	task-level SLOs, capacity prediction and cost attribution across heterogeneous resources

“Task envelope” is the report's name for the identity, budget, environment, state and evidence that belong to one run and should share a lifecycle. The table aligns existing projects with that task and makes the remaining seams visible: today those controls live in different systems and stop at component boundaries.

The workload is bursty before it is simply “large”

Agent workloads vary at task level. One run may make a serial model call, fan out to several tools, retry after a timeout and then wait for approval. Two tasks with the same total token volume can therefore create very different peaks, failure amplification and cost. Public platform totals confirm demand, while their aggregation hides this operational shape.

Agentgateway already exposes request and token limits, plus per-tool limits for MCP. Its documentation also states that local counters are not exact global limits and do not survive a restart. That leaves an open systems problem: a task budget has to remain valid across gateways, model calls, tools and execution environments, and cancellation has to reach the work already in flight.

Recovery becomes harder after a tool has changed the outside world

Dapr Agents packages durable workflows, retries, failure recovery and persistent state as production capabilities. This confirms that long-running and interrupted tasks are already an infrastructure concern. The remaining difficulty appears when a failed step has changed another system. Retrying may create the same resource twice, send the same message twice or repeat a deployment action.

Open workflow systems already provide useful machinery. Agent runtimes still need a shared way to distinguish replayable computation from an action that requires an idempotency key, compensation, external verification or human approval.

Sandboxes become runtime objects

The current Agent Infra selection contains four development sandbox projects. Kubernetes Agent Sandbox exposes Sandbox, SandboxTemplate, SandboxClaim and SandboxWarmPool. It can use gVisor or Kata Containers for stronger isolation.

The scheduling object is the important change. The platform manages a short-lived session with identity, storage, network policy, warm capacity and an expiry time. The code may not exist when the surrounding application is deployed.

Telemetry has to reach the effect

An HTTP success code ends at the request boundary. To explain whether the Agent changed the intended file or sent the intended message, the trace has to connect model work, tool execution, sandbox events and the external result. OpenTelemetry provides the pipeline, and its GenAI semantic conventions already cover Agents and tool execution, with parts of that semantic layer still in Development.

The useful trace begins before the tool call and ends at the external effect.

Open-source Collaboration

02A • Workload and repository setting

The study freezes the 100 highest-OpenRank repositories in the 277-project tracking pool. OpenRank decides the sample and does no further analytical work. Each repository was then reviewed as `llm_native`, `traditional` or `mixed`, with a confidence level and a short reason.

The review produced 68 LLM-native projects, 18 traditional projects and 14 mixed projects. `Mixed` means the repository still has a complete non-LLM purpose, while AI or agents have become a substantial product surface; `n8n`, `Warp` and `MLflow` fall into this group. A binary creation-date rule would miss 19 projects: 14 genuinely span both worlds, and five directly contradict the date proxy. `LangChain`, `Megatron-LM` and `TRL` predate `ChatGPT` but were built around language models. `ComfyUI` and `Apache Gravitino` were created later, yet their core value does not depend on an LLM.

What is in the Top 100?

Lens	Distribution
Technical role	36 Model Infra · 28 Agent applications · 21 Agent frameworks · 15 Agent runtime infrastructure
Project identity, manually reviewed	68 LLM-native · 18 traditional · 14 mixed
Repository creation	72 created in December 2022 or later · 28 created earlier
GitHub primary language	44 Python · 26 TypeScript · 11 Go · 19 other languages

The sample is the most active slice of the tracked ecosystem by July OpenRank. Its repository scale, release cadence and contribution volume describe prominent Agentic AI projects; smaller and quieter repositories sit outside this view.

One repository frame, two layers of evidence

The report starts from the same frozen Top 100 throughout. It then uses the data at two different depths.

Evidence layer	Coverage	What it answers
Complete repository counts	Every public Issue and pull request that GitHub Search finds in the Top 100 for January–August 2024, 2025 and 2026; historical comparisons use the same 55 repositories with activity in every fixed window	How much work arrived, how much was closed, what remained open after a fixed 90-day period, and whether the queue grew
Repository-balanced thread timelines	50 Issues or pull requests per repository in 2026; the historical panel uses 2,750 threads from the same 55 repositories in each of 2025 and 2026	Who responded after the opener, how quickly a repository-team account appeared, and how many visible review-and-revision rounds followed

The complete counts carry the workload and outcome claims. The thread timelines explain how that work moved through public collaboration. The ten code-lineage pull requests and seven readable public threads are closer views drawn from the 2026 sample.

Pull requests are arriving faster than issues

Between 1 January and 31 August 2026, the Top 100 opened about 349,800 Issues and 606,700 pull requests. That is 1.73 pull requests for every Issue. The monthly ratio rose from 1.35 in January to 2.11 in the complete month of August.

Month	Issues opened	Pull requests opened	PR / Issue
January	26,320	35,540	1.35×
February	35,543	50,329	1.42×
March	49,896	74,831	1.50×
April	50,167	71,173	1.42×
May	44,040	78,753	1.79×
June	41,604	83,082	2.00×
July	49,274	101,482	2.06×
August	52,982	111,551	2.11×

The historical comparison keeps repository membership fixed. Fifty-five of the current Top 100 have public Issue or PR activity in the January–May fixed-maturity cohort in all three years. Those same 55 repositories are used for queue flow, 90-day outcomes and PushEvent concentration.

Same 55 repositories · January–August	Issues opened	Issues closed	Issue balance	PRs opened	PRs closed	PR balance
2024	53,847	46,033	+7,814	97,793	96,367	+1,426
2025	74,586	64,001	+10,585	129,563	125,104	+4,459
2026	71,539	72,789	−1,250	265,447	222,675	+42,772

“Balance” is opened minus closed during the eight-month window. A positive number means more work arrived than the repositories closed; closures may include older backlog. In 2026, the Issue side was almost balanced, while PR intake doubled and exceeded closures by 42,772. Fifty-four of the 55 repositories added to their PR queue. This is not a general collapse in issue handling. The pressure is concentrated in proposed code changes that still need review, revision and a merge decision.

The result remains after giving every item a comparable amount of time. For the January–May cohorts, each Issue and pull request is observed for 90 days after its month ends.

Same 55 repositories · fixed 90-day outcome	2024	2025	2026
Issues still open after 90 days	32.3%	31.7%	28.6%
PRs still open after 90 days	4.6%	5.5%	11.3%
Repository-median PR merge rate by 90 days	81.1%	77.0%	68.4%

The contrast matters. Issue closure improved modestly; PRs became less likely to reach a recorded merge within the same 90-day window and twice as likely to remain open. The 2026 backlog is not only a pile of late-August submissions that had no time to move.

The four technical roles do not carry that pressure in the same way.

2026 technical role	Issue queue balance	Repositories with growing Issue queue	PR queue balance	Repositories with growing PR queue	Median PR still open after 90 days	Median PR merged by 90 days
Agent applications	+45,865	21 / 28	+54,625	26 / 28	7.0%	56.6%
Agent frameworks	+7,407	13 / 21	+17,139	21 / 21	8.7%	68.1%
Agent runtime infrastructure	+526	13 / 15	+5,715	15 / 15	6.9%	73.7%

2026 technical role	Issue queue balance	Repositories with growing Issue queue	PR queue balance	Repositories with growing PR queue	Median PR still open after 90 days	Median PR merged by 90 days
Model infrastructure	+321	29 / 36	+35,735	36 / 36	11.4%	67.8%

Agent applications accumulated both unanswered questions and unprocessed code changes. Infrastructure repositories kept the aggregate Issue flow much closer to balance, yet every framework, runtime and model-infrastructure repository added to its PR queue. Model infrastructure has the highest repository-median 90-day unresolved PR share, while Agent Runtime has the highest median merge rate in the fixed window.

The Top 100 flow is concentrated. Claude Code, OpenClaw, Hermes Agent, OpenCode and Codex account for 54.5% of Issue intake, while the five PR leaders account for 34.7% of pull requests. The report therefore keeps aggregate totals beside repository medians and counts of affected repositories.

Agent activity reached more PRs, while fewer finished within 30 days

The complete repository counts above show the growing queue. To see what happened inside it, we compared 2,750 threads from January–August 2025 with 2,750 from the same 55 repositories in 2026. Each repository contributes 50 threads in each year. A maintainer response counts only when a repository Owner, Member or Collaborator other than the opener responds from a GitHub User account.

Same 55 repositories · 50 sampled threads per repository and year	2025	2026	Change
Threads where a named Agent or App appeared	19.9%	46.1%	+26.3 pp
A repository maintainer responded within 7 days	37.1%	31.1%	−6.0 pp
Pull requests resolved within 30 days	87.8%	80.3%	−7.5 pp
Pull requests with a visible review	68.8%	73.9%	+5.1 pp

Named Agents appeared in more than twice as many threads, and review reached a larger share of PRs. At the same time, first-week maintainer response and 30-day PR completion both fell. Agents helped more changes reach review; repositories still had to find the attention and authority to finish them.

Some repositories publish GitHub Releases almost every day

From 1 January to 31 August 2026—a 243-day window—98 repositories published at least one non-draft GitHub Release. A release day is a UTC date with at least one such record. The median repository published on 34 distinct days; the middle half spans 15 to 103 days, and six repositories published on at least 180 days.

Release records also cover 1 January–31 August 2026. Vercel AI's 15,232 records fall on 194 dates, reflecting a multi-package and canary pipeline. llama.cpp published 2,041 records across 241 of the 243 days. At this frequency, release days reveal automated delivery cadence more than occasional milestone launches. Tag-only releases and versions published only to PyPI, npm or another registry sit outside the GitHub Release record.

Release days in the 243-day window	Repositories
None	2
1 day	2
2–9 days	13
10–29 days	26
30–89 days	27
90–179 days	24
180+ days	6

Repository	Release days	GitHub Release records
ggml-org/llama.cpp	241 / 243	2,041
QwenLM/qwen-code	222 / 243	492
openai/codex	208 / 243	681
router-for-me/CLIPProxyAPI	203 / 243	440
vercel/ai	194 / 243	15,232
flashinfer-ai/flashinfer	185 / 243	221

The core integration circle widened, but PR intake grew faster

PushEvents show how widely the final write path is shared. For each repository, we ranked public PushEvent accounts and counted how many were needed to cover half of all pushes. In the same 55 Agentic AI repositories, the median number of accounts with a PushEvent rose from 13 in 2024 to 17 in 2025 and 25 in 2026. The median number producing half of all pushes rose from two to three.

Same 55 Agentic AI repositories	Median PushEvent accounts	Accounts producing half of pushes	Median share produced by top five accounts
2024	13	2	91.4%
2025	17	2	86.9%
2026	25	3	74.1%

The core did not shrink. More accounts entered the public integration path, and work became less concentrated in the top five. But from 2025 to 2026, the median PushEvent circle grew 47%, while PR intake in the same repositories grew 105%.

OpenDigger technology labels provide two active-repository comparisons. In 2026, an Agentic AI repository needed a median of three accounts to produce half its pushes, compared with two in the cloud-native benchmark and one in the big-data benchmark.

2026 benchmark	Active repositories	Median PushEvent accounts	Accounts producing half of pushes
Agentic AI matched panel	55	25	3
Cloud Native	98	12	2
Big Data	56	7	1

A PushEvent account is the account that wrote to the repository, not necessarily the commit author. The comparison is useful because it shows how many public accounts share the integration path. Agentic AI projects have a broader core than the two established technology controls; the current bottleneck is not explained by a core that is simply getting smaller.

02B · Repository access and Agent setup

Most repositories accept outside pull requests

All 100 repositories have Issues and Pull Requests enabled. The creation setting is not identical: 98 allow anyone to create a pull request, while Codex and Claude Code restrict creation to collaborators. Discussions are enabled in 74. A common-path scan found a CONTRIBUTING file in 89 repositories, an Issue template in 95 and a pull-request template in 84.

FIGURE 10

Current collaboration surface of the Top 100

100 / 100 Issues enabled	100 / 100 Pull Requests enabled	98 / 100 Anyone can create a pull request	2 / 100 Creation restricted to collaborators
74 / 100 Discussions enabled	89 / 100 CONTRIBUTING found	95 / 100 Issue template found	84 / 100 Pull-request template found

GitHub's repository settings answer whether Pull Requests are enabled and who can create one. We combine that with the contribution policy and sampled outcomes because opening access, declared expectations and eventual merge behavior describe three different parts of the contribution surface.

We therefore read the API settings first, froze common contribution documents, and manually reviewed every candidate phrase that looked restrictive. Forty-eight repositories explicitly invite contribution. Twelve ask for an Issue first, prior agreement or a change within a stated scope. Thirty-eight contain no restrictive signal in the reviewed files. The remaining two — Codex and Claude Code — leave Pull Requests visible but set creation access to collaborators only.

The distinction is visible in individual repositories. Mastra asks code contributors to open an Issue before a Pull Request. Open WebUI applies that gate to first-time contributors except for localization work. Repositories enter “No restrictive signal detected” when the reviewed settings and documents contain neither an explicit invitation nor a stated gate.

DeepSeek Harness sits outside this Top 100 denominator. Its MIT-licensed core keeps Issues and Pull Requests closed while directing outside development toward plugins. It is a useful comparison because open source code, an open core contribution path and an extension ecosystem are separate choices.

Coding-agent setup is already common across the stack

In 92 of the Top 100 repositories, the default branch contains something created for a coding-agent workflow. This includes instruction files such as `AGENTS.md` and `CLAUDE.md`, as well as tool folders such as `.claude`, `.cursor`, `.codex` and `.gemini`.

We count both forms as repository-level adoption. The project has added files or folders specifically for a coding agent, so the report presents them as one measure.

The current files show a clear hierarchy. Eighty repositories publish instructions that more than one coding agent can follow. Seventy-one include Claude Code files, 22 Codex, 20 GitHub Copilot, 17 Cursor and 12 Gemini. A repository can support several agents at once.

Files found on the default branch	Repositories
Instructions that work across coding agents	80 / 100
Claude Code	71 / 100
Codex	22 / 100
GitHub Copilot	20 / 100
Cursor	17 / 100
Gemini	12 / 100

LobeHub, Opik, Cline and OmniRoute each publish files for four agent-specific formats. The count measures compatibility work that maintainers have committed to the default branch: one project may support several tools even when its contributors use them at very different frequencies.

The full annual scan finds Cursor files or folders in 13 repositories in 2025 and 17 in 2026. The current 92% adoption count requires an instruction file or tool-specific folder on the default branch; `.gitignore` residue is excluded. This keeps the measure tied to repository content that can actually configure or guide the tool.

The rules have moved into Model Infra

Coding-agent files or folders appear in 20 of 21 Agent Framework repositories, all 15 Agent Runtime Infra repositories and 32 of 36 Model Infra repositories.

FIGURE 11

Coding-agent setup by technical niche



PyTorch, Spark, Iceberg, ONNX Runtime, Milvus, Triton and OpenVINO all carry machine-readable instructions in the current snapshot. The change is therefore wider than the repositories that sell an Agent experience directly.

The instruction text also reaches beyond implementation. Among the 86 repositories where we could read explicit instructions, 81 mention tests or validation, 79 mention Issue work or planning, 72 mention code review and 63 mention release or dependency work. Maintainers are preparing Agents for the full contribution loop, especially the checks and coordination that happen around code generation.

02C · Where Agents enter the public workflow

The 5,000-thread sample follows public collaboration in detail

We sampled 50 non-overlapping threads from each repository, giving every project an equal place in the study instead of letting the largest repositories dominate. The final sample contains 5,000 threads: 1,433 Issues and 3,567 pull requests.

Every sampled thread counts once. Taking the same number from each repository prevents the largest projects from swallowing the rest of the sample and keeps all 100 repositories visible in the result. The figures therefore describe an equal-sized slice of each repository: 5,000 threads drawn from the 956,567 Issues and pull requests opened during the eight-month window.

Agents mostly join after a contribution arrives

A coding or review Agent left a visible action in 2,158 of the 5,000 Issues and pull requests we reviewed. We saw this in 95 of the 100 repositories. Only 87 threads were opened by an Agent account or App. Put plainly: Agent participation appears in 43.16% of this sample, while Agent-opened work accounts for 1.74%.

On GitHub, this usually looks like CodeRabbit reviewing a pull request, Gemini Code Assist leaving review comments, or OpenHands acting through its GitHub App. An Agent responds after the opener in 38.28% of all sampled threads and appears in review events in 37.62% of sampled pull requests. Agents are entering the contribution process where repositories already spend attention: triage, discussion and review.

How we counted visible Agent activity

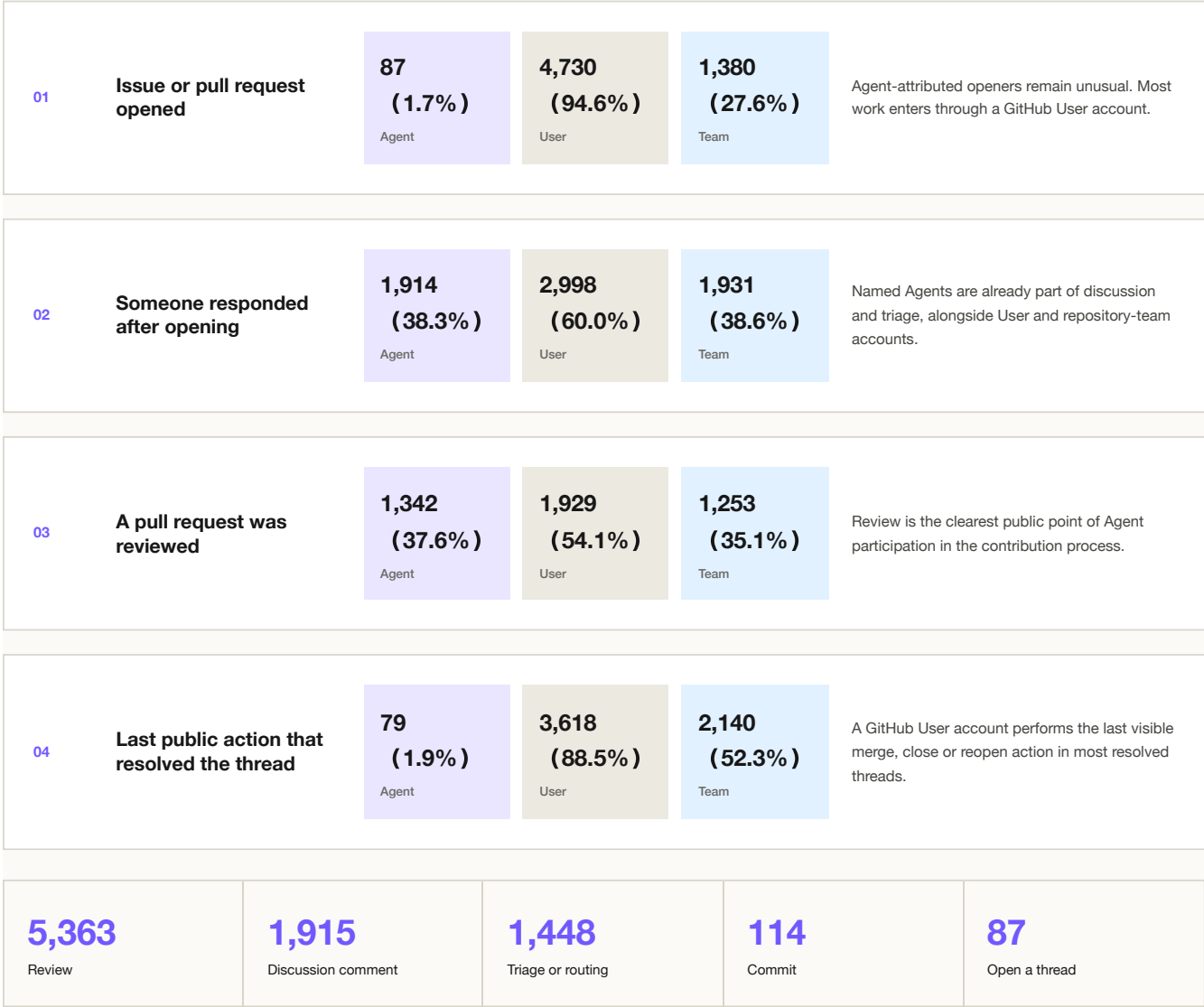
We counted an action only when GitHub named a known coding or review Agent, exposed the GitHub App behind the action, or the contribution explicitly said it was Agent-generated. Dependabot, GitHub Actions, release bots and other conventional automation are classified separately.

The registry behind this count contains 63 named identities tied to coding, review, security review, support or App-mediated Agent work. Fifty-two have direct identity or App evidence; eleven rely on documented function. The same login can appear through more than one GitHub actor type, so Bot and User observations are not added together as separate people.

In this report, a visible Agent action is one where GitHub names the Agent or the App behind it, or where the contribution explicitly attributes the work to an Agent. Local use of Cursor, Claude Code or Codex stays under the developer's ordinary User account, so those contributions remain in the User column. Wording, timing and code style are not used to reclassify them.

FIGURE 12A

Who appears at each stage of a public Issue or pull request



GitHub User account means GitHub reports the actor as account type User. Local tool use usually remains under that account because GitHub exposes no separate Agent identity. Repository team account means GitHub associates the account with the repository as OWNER, MEMBER or COLLABORATOR. The columns can overlap: an App may mediate a User action, and a repository-team account is usually also a User account.

The final row uses the latest visible merge, close or reopen event in a thread that was resolved when collected. It shows which account executed the public state change. The earlier rows show who opened, responded and reviewed, so the table can be read as a hand-off sequence rather than a single attribution of the whole contribution.

The event mix makes the pattern concrete. Named Agent identities produced 5,363 review events, 1,915 discussion comments and 1,448 triage or routing events, compared with 87 thread-opening events and 114 publicly attributed commit events. The unit is a public event, so a single thread can contribute several reviews or replies. Even with that repetition, the distribution shows where Agent services are being used most heavily.

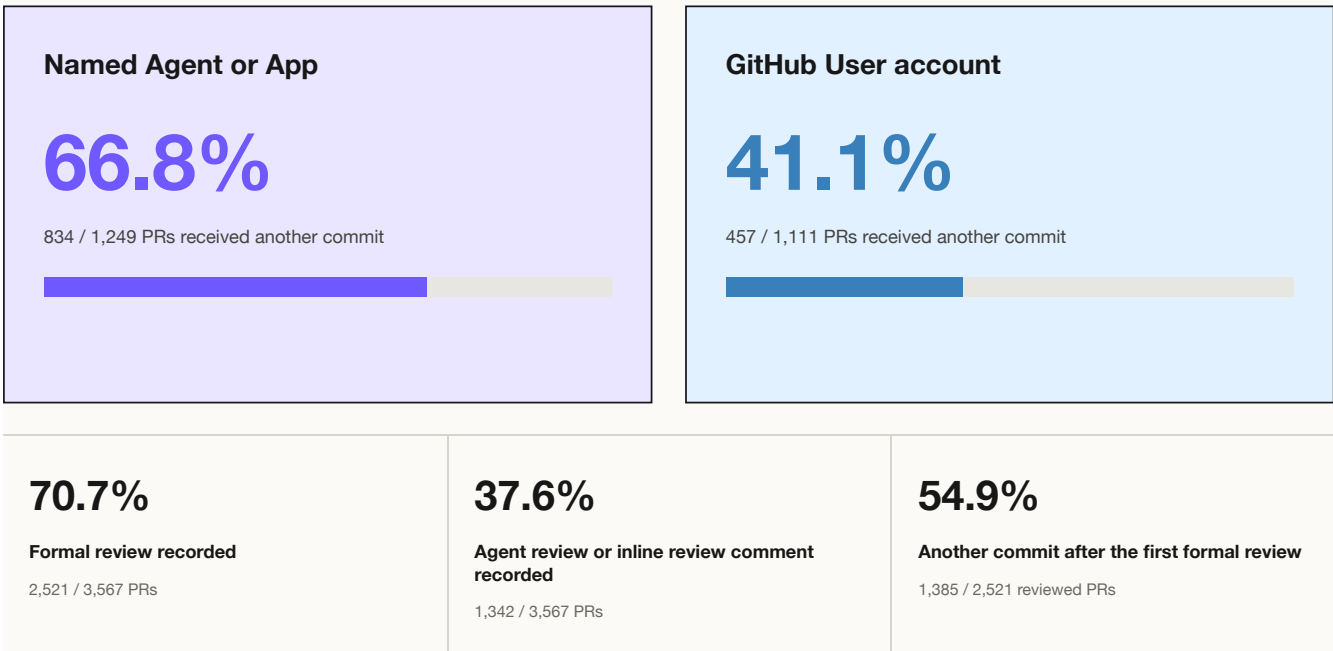
Agent services are only one part of repository automation. GitHub Actions, project automation, Codecov, merge queues and dependency bots remain the most widely visible conventional layer. Coding Agents include Copilot, Codex, Cursor, Claude, Devin, Gemini CLI, Kilo Code and Warp service identities; review Agents include CodeRabbit, Gemini Code Assist, Greptile and similar services; support and security roles include Dosu, automated triage and security-review Apps. Identity and functional role are preserved separately in the actor registry.

The first reviewer changes what happens next

We ordered the formal reviews and commits in all 3,567 sampled pull requests by time. Formal review appeared in 2,521 PRs, Agent review or inline review comments appeared in 1,342, and 1,385 reviewed PRs received another commit after the first review.

FIGURE 12B

Later commits by first formal reviewer



When the first formal review came from a named Agent or App, 834 of 1,249 PRs received another commit. When it came from a GitHub User account, 457 of 1,111 did. The public sequence shows Agent review sitting

inside an active revision loop: code changes continue after it more often in this sample.

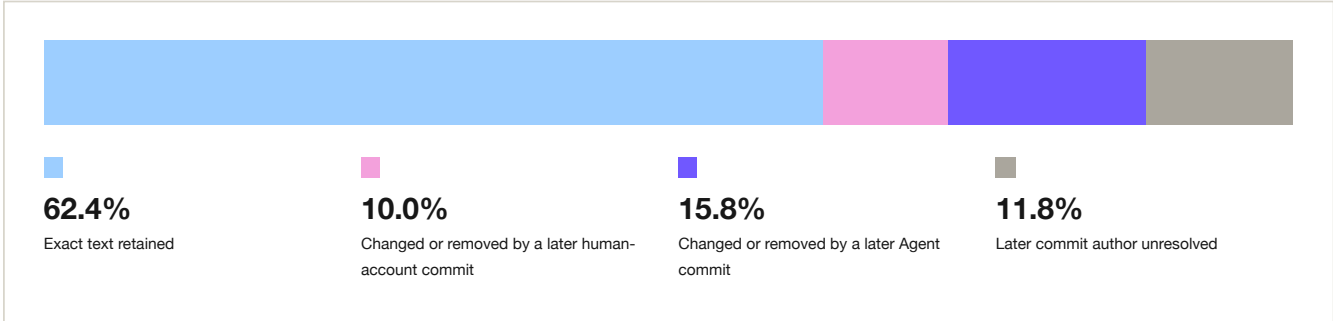
The first Agent patch often survives—and sometimes gets rewritten

We followed ten merged pull requests in which a verified Coding Agent opened the contribution or authored a commit. Nine expose a clean line history from the first effective Agent patch through the final PR head. The tenth, [Mooncake #2686](#), attaches the Agent identity to a two-parent merge commit whose first-parent diff includes thousands of upstream lines, so it remains in the casebook while its lines stay outside the denominator.

Across the nine traceable PRs, the first effective Agent commits added 1,225 text lines. Of those, 765 lines — 62.4% — remain as exact text in the final PR head. Later human-account commits first changed or removed 123 lines; later Agent commits changed 193; 144 were changed by commits without a resolvable GitHub author.

FIGURE 12B.1

What happened to the first observable Agent patch



Pull request	First Agent patch	Exact retained	Human-account change	Later Agent change	Unresolved	Path
vercel/ai #18818	172	0	0	172	0	Agent iterates to merge
warpcdotdev/warp #13382	44	31	12	1	0	Agent → human
OpenMetadata #25243	62	21	29	12	0	Agent → human
ONNX Runtime #28045	611	533	78	0	0	Agent → human
OpenHands #2614	11	0	4	7	0	Agent → human
MLflow #19721	262	118	0	0	144	Agent → unresolved author
MLflow #21621	33	33	0	0	0	Agent iterates to merge
MLflow #22355	25	25	0	0	0	Agent → human
MLflow #22659	5	4	0	1	0	Agent iterates to merge

Five cases follow a visible Agent-to-human code handoff. Their pooled line retention is 81.0%, although ONNX Runtime contributes 611 of the 753 starting lines. The case-level median is more useful here: 70.5% of the first patch survives unchanged, while 27.3% is later changed by a human account. The five cases range from no exact lines retained to the entire first patch retained.

The line history reveals several different hand-offs. In ONNX Runtime #28045, 533 of 611 first-patch lines survive while later User-account commits change 78. In Vercel AI #18818, later Agent commits replace all 172 first-patch lines. A merged diff alone hides whether the first patch survived, was refined by a person or was rewritten by another Agent.

The unit is an exact text line added by the first attributable Agent commit that contains code; empty planning commits are skipped. Each line is carried through subsequent PR commits, and the first commit that changes or removes it receives the public account attribution. Git blame resolves duplicate-line alignment. This makes the comparison reproducible at text level. Questions about semantic authorship, code importance and undisclosed AI use remain in the methodology section.

The outcome trace later returns to actual submissions. The historical window contains at least one external-association PR in 99 of 100 repositories. The current setting shows who may open a PR today; the sampled submissions show who actually did so during the study window. The expanded sample reports the observed share of PRs opened by external accounts directly, with every sampled thread counting once.

DeepSeek Harness makes a different governance choice

DeepSeek Harness sits outside the Top 100 and serves as a contrasting governance case. The repository was opened on 13 August 2026 and reached 204,176 GitHub stars and 23,597 forks by 30 August, seventeen days later. The code is released under MIT and Discussions are open. Issues are disabled, the Pulls endpoint returned 404 in two checks, and the contribution guide directs community development towards third-party plugins while keeping core pull requests closed for now.

FIGURE 12C

DeepSeek Harness contribution surface

Source code	Issues	Pull requests	Discussions	Extension path
Public · MIT	Off	Off	On	Plugins

Open code, open core contribution and a wider plugin ecosystem are three separate governance choices.

Seven public threads show what the hand-offs look like

The interactive casebook contains [Coder #25800](#), [ONNX Runtime #28045](#), [LangChain #37607](#), [PyTorch #182986](#), [Supabase #42193](#), [Gemini CLI #24026](#) and [n8n #33411](#). Each has a readable public sequence showing who opened the work, where an Agent entered, who revised it and who closed the loop. The cases explain the hand-offs hidden behind a simple merged, closed or fixed label.

GitHub User accounts respond after the opener in 60.1% of sampled threads, and maintainer-associated accounts respond in 38.7%. In 27.6%, every visible response after the opener comes from automation; only 0.56% contain no visible User account anywhere in the thread. Automation is taking a larger share of routing and response work, while User accounts remain present where exceptions are handled and repository state changes.

Outside contributors supply most pull requests. User accounts still perform the final public action

External accounts create 66.8% of the sampled pull requests. At the fixed-maturity checkpoint, GitHub's merged flag appears on 57.2% of resolved external PRs (95% within-sample interval 54.7–59.7%) and 82.8% of resolved maintainer or member PRs (80.2–85.3%). Repository access and context still shape which changes move through the gate, even when the initial code supply is broad.

Agents expand the supply of patches. Open source still decides what a project can absorb and maintain

The public record shows a new division of work. Repositories increasingly publish instructions that an Agent can follow. Agents review, triage, discuss and revise contributions across most of the sample. Outside accounts still supply most pull requests, and GitHub User accounts perform most of the last visible actions that merge, close or reopen the work.

The matched evidence sharpens that picture. PR supply doubled and review reached a larger share of sampled PRs, but early human response and timely PR completion fell. Issue flow, by contrast, was close to balanced. Agents are adding production and review capacity; they have not yet removed the constraint around judgment, integration and long-term ownership. The line-level cases show why: a first Agent patch may survive, be refined by a person or be replaced by another Agent before it becomes code the project will carry.

The next useful measure begins after the repository gate: whether an accepted change is reverted, needs follow-up fixes, brings the contributor back and continues to hold up in tests or benchmarks.

In the Agent era, a patch is the beginning of a contribution. Contribution also means understanding a problem the project recognizes, working within its rules, responding to review and leaving behind code that other people are willing to maintain. **Agents can expand the supply of code; open-source collaboration determines how much of it a project can responsibly absorb and carry forward.**

Method and evidence boundaries

- The current project list comes from `data/agent-ai-projects.csv`.
- The May baseline comes from `data/history_snapshot/2605_agentic_projects.csv`.
- OpenRank uses the complete July 2026 month.
- Stars and GitHub primary-language labels use the canonical project snapshot updated on 23 August 2026.
- OpenRouter App & Agent rankings are public and opt-in. The Top 20 alignment was checked on 29 August 2026 and will move as platform traffic changes.
- ZenMux figures use a frozen single-platform export for 1–30 June 2026. Weight access means an official public weight repository was resolved; it is not an OSI license determination. The values are not combined with OpenRouter traffic.
- Agent Sandbox, Kata Containers, kagent, Dapr Agents, OpenChoreo, kgateway, agentgateway, Istio, OpenTelemetry and Jaeger are cited as project-level evidence of engineering work around agent execution. Confidential Containers is labelled as an adjacent AI substrate. Their documentation does not establish how widely those capabilities are deployed.
- Collaboration figures use a frozen Top 100 and 5,000 threads from January–August 2026: 50 non-overlapping Issues or pull requests per repository. Each thread counts once; the report does not reweight repositories by traffic.
- Historical flow and 90-day outcomes use GitHub's full repository counts for the same 55 repositories that have comparable activity in 2024, 2025 and 2026. The response and revision comparison uses 2,750 sampled threads in 2025 and 2,750 in 2026 from those same repositories.
- PushEvent concentration uses OpenDigger's ClickHouse event data. Cloud-native and big-data comparisons use OpenDigger technology labels and active repositories with July 2026 OpenRank data.
- Public actor labels separate verified Agent services, conventional automation, App-mediated User actions and GitHub User account types. They cannot observe undisclosed local AI use.
- Review-to-commit sequence uses dedicated PR commit timestamps. Timeline commit rows without a timestamp are not interpreted as no later revision.
- GitHub's merged flag is reported as an observable gate signal, not a universal accepted-contribution or quality label.
- The May tracking pool is not identical to the published May map. Exact add, remove and reclassification claims require a reconstructed machine-readable May map inventory.
- GitHub attention, open-source collaboration and production adoption remain separate labels throughout the report.
- DeepSeek Harness repository settings and contribution guidance need a final publication-date recheck.

Detailed study design: `../research/open-collaboration-study-design.md` Landscape evidence and chart map:

`../research/landscape-signals.md` Open infrastructure evidence: `../research/open-infrastructure-trends.md`

References

- [GitHub REST API documentation](#)
- [GitHub GraphQL pull request types](#)
- [GitHub REST API endpoints for timeline events](#)
- [GitHub issue and pull-request search qualifiers](#)
- [OpenRank metric documentation](#)
- [OpenDigger data description](#)
- [OpenDigger labeled data](#)
- [OpenRouter App & Agent Rankings](#)
- [ZenMux App Leaderboard API](#)
- [ZenMux Model Leaderboard API](#)
- [Agent Sandbox quickstart](#)
- [Agent Sandbox threat model](#)
- [Kata Containers Agent Sandbox integration](#)
- [OpenInfra Foundation projects](#)
- [Deploying the SPIRE Agent](#)
- [OpenTelemetry GenAI agent span conventions](#)
- [Kagent: Bringing Agentic AI to Cloud Native](#)
- [Dapr Agents v1.0](#)
- [OpenChoreo and the agentic enterprise](#)
- [Kgateway v2.1 and agentgateway](#)
- [Istio in the AI era](#)
- [Jaeger tracing AI agents with OpenTelemetry](#)
- [Confidential Containers becomes a CNCF incubating project](#)
- [Kubernetes v1.34 Dynamic Resource Allocation updates](#)
- [Kubernetes Agent Sandbox roadmap](#)
- [Kueue overview](#)
- [Kueue topology-aware scheduling](#)
- [Agentgateway request and token rate limits](#)
- [Agentgateway MCP per-tool rate limits](#)
- [DeepSeek Harness repository](#)
- [DeepSeek Harness contribution guide](#)
- [OpenViking repository](#)
- [Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity](#)
- [We are Changing our Developer Productivity Experiment Design](#)
- [AIDev: Studying AI Coding Agents on GitHub](#)
- [AIDev open dataset](#)
- [On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub](#)
- [Where Do AI Coding Agents Fail?](#)
- [From Industry Claims to Empirical Reality: An Empirical Study of Code Review Agents in Pull Requests](#)
- [Security in the Age of AI Teammates](#)
- [Understanding the Rejection of Fixes Generated by Agentic Pull Requests](#)

- [AI Agent Pull Requests on GitHub: Frequency, Structure, and Merge Conflict Rates](#)
- [State of Open Source AI](#)